Orbit Platform - Architecture Knowledge Base

Internal engineering reference. Covers architecture, auth, data layer, deployment, incident response and key decisions.

1. Architecture Overview

Orbit is an order-processing platform running on AWS in the eu-west-1 region. All services run as containers on Amazon EKS (Kubernetes).

Services

- **api-gateway**: Envoy proxy. Single entry point for all external traffic.
- **auth-service**: written in Go. Handles login and tokens.
- **order-service**: written in Python (FastAPI). Creates and tracks orders.
- **payment-service**: written in Java (Spring Boot). Talks to the external payment provider.
- **inventory-service**: written in Python. Reserves stock.
- **notification-service**: written in Node.js. Sends email and SMS.

Communication

- Gateway to services: synchronous REST over HTTP.
- Service to service events: asynchronous, through Apache Kafka.
- Kafka topics: order.created, inventory.reserved, payment.completed, order.shipped.

Flow of a new order

1. Client calls api-gateway.
2. order-service stores the order and publishes order.created.
3. inventory-service reserves stock and publishes inventory.reserved.
4. payment-service charges the customer and publishes payment.completed.
5. notification-service sends a confirmation to the customer.

2. auth-service

Purpose

Issues and validates tokens for all Orbit clients using OAuth2 and JWT.

Tokens

- Access token lifetime: 15 minutes.
- Refresh token lifetime: 7 days.
- Signing algorithm: RS256.
- Signing keys are rotated every 90 days.

Sessions

- Session data is stored in Redis 7.
- The service listens on port 8081.

Security rules

- After 5 failed logins within 10 minutes, the account is locked for 15 minutes.
- Passwords are hashed with bcrypt.
- All endpoints require TLS 1.2 or higher.

Dependencies

- Redis for sessions.
- PostgreSQL database named auth_db for user accounts.

3. Data Layer

PostgreSQL

- Version: PostgreSQL 15, managed with Amazon RDS.
- Topology: 1 primary and 2 read replicas.
- Each service owns its own database: auth_db, orders_db, payments_db, inventory_db.
- Services must never read another service's database directly.
- Maximum connection pool size: 50 per service.

Backups

- Automated backup runs nightly at 02:00 UTC.
- Backups are kept for 30 days.
- Point-in-time recovery is available for the last 7 days.

Redis

- Used for session storage (auth-service) and caching (order-service).
- Default cache TTL: 300 seconds.
- Redis is not a source of truth. Data in Redis can be lost without data loss for customers.

4. Deployment Runbook

Pipeline

- CI: GitHub Actions runs tests and builds the container image.
- CD: ArgoCD deploys to Kubernetes.
- Order of environments: dev, then staging, then production.

Production rollout

- Production uses canary releases.
- A canary receives 10% of traffic for 15 minutes.
- If error rate stays below 1%, the rollout continues to 100%.
- Production namespace: orbit-prod.

Rollback

- Command: argocd app rollback <app-name>
- Rollback typically completes in under 3 minutes.

Rules

- Deploy freeze: no production deployments on Fridays after 14:00 UTC.
- Every service must expose a health endpoint at /healthz.
- Database migrations must be backward compatible with the previous release.

5. Incident Response

Severity levels

- **SEV1**: full outage or data loss. Response within 5 minutes. Page the on-call engineer and the engineering manager.
- **SEV2**: major feature degraded. Response within 30 minutes.
- **SEV3**: minor issue with a workaround. Response by the next business day.

On-call

- Tool: PagerDuty.
- Rotation: weekly. The handover happens every Monday at 09:00 UTC.

After an incident

- A written postmortem is due within 5 business days.
- Postmortems are blameless.
- Customer-facing incidents are posted on the public status page.

First steps for any incident

1. Acknowledge the page.
2. Open an incident channel in Slack named #inc-YYYYMMDD.
3. Check recent deployments, since most incidents follow a release.

6. ADR-007: Message Broker Selection

Status: Accepted **Date:** 2025-03-12

Context

Orbit needs asynchronous communication between services. RabbitMQ was used in the first prototype.

Decision

We use Apache Kafka and have removed RabbitMQ.

Reasons

- Messages can be replayed, which helps recover after a consumer bug.
- Higher throughput than RabbitMQ at our expected volume.

Configuration

- Retention: 7 days.
- Each topic has 3 partitions.
- Replication factor: 3.

Consequences

- The team must operate Kafka, which is more complex than RabbitMQ.
- Consumers must handle duplicate messages (at-least-once delivery).